When $i > m$, let

$$x_{i,j} = \begin{cases} 0, & j \leq 2 \\ -1, & j > 2,\ i \equiv j \pmod{h} \\ 0, & j > 2,\ i \not\equiv j \pmod{h}. \end{cases}$$

Finally, let $y_i = 1$ when $i \leq l$ and 0 otherwise.

$$X = \begin{array}{c} \\ \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \vdots \\ \mathbf{x}_k \\ \vdots \\ \vdots \\ \mathbf{x}_l \\ \vdots \\ \vdots \\ \mathbf{x}_m \\ \vdots \\ \vdots \\ \mathbf{x}_n \end{array} \left[\begin{array}{cc|cccc} \mathbf{f}_1 & \mathbf{f}_2 & \mathbf{f}_3 & \mathbf{f}_4 & \dots & \mathbf{f}_{h+2} \\ 1 & 0 & 1 & -1 & \cdots & -1 \\ 2 & 1 & -1 & 1 & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & -1 \\ \vdots & \vdots & -1 & \cdots & -1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \hline \vdots & \vdots & & & & \\ l & l-1 & \vdots & \ddots & I_h & \ddots \\ \hline & -I_2 & & & 0 & \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ \hline & 0 & & & -I_h & \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots \end{array}\right] \quad \mathbf{y} = \begin{array}{c} y_1 \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ y_l \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ y_n \end{array} \left[\begin{array}{c} 1 \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \hline 0 \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \end{array}\right]$$

Figure 4: Our dataset.

From the fact that $\phi(\mathbf{f}_1) - \phi(\mathbf{f}_2) = \sum_{j=3}^{h+2} \phi(\mathbf{f}_j) = \mathbf{y}$ it follows that there exist networks with both two active hidden units and h active hidden units that achieve minimal loss, with the former corresponding to the ground truth teacher network which generated our dataset.

Note in particular that the output layer weight corresponding to $\phi(\mathbf{f}_2)$ in the network with two active hidden units is negative, whereas in the network with h active hidden units the output layer weights are all positive. Thus, any path between the two networks must pass through a point in parameter space where at least one output layer weight is zero while the other $h-1$ are positive. However, as shown in Lemma 6, there does not exist such a point in parameter space that achieves minimal loss. It follows that there exists a barrier in the loss landscape separating the original networks, both of which are global minima. Moreover, by adjusting k, l, and m we can somewhat arbitrarily raise or lower this barrier. □

Lemma 6. *There does not exist a set of $h-1$ positive weights w_i and vectors $\mathbf{h}_i \in \operatorname{span} X$ such that $\sum_{i=1}^{h-1} w_i \phi(\mathbf{h}_i) = \mathbf{y}$.*

Proof. We can think of each $\mathbf{h}_i$ as the output a particular hidden unit over all n samples in our dataset and w_i as the output layer weight associated to this hidden unit. We then have $\mathbf{h}_i = \sum a_{i,j} \mathbf{f}_j$, where the coefficients $a_{i,j}$ are elements of A.

First, if there did exist w_i and $\mathbf{h}_i$ such that $\sum_{i=1}^{h-1} w_i \phi(\mathbf{h}_i) = \mathbf{y}$, then it must be the case for all i that $\mathbf{h}_i = \sum a_{i,j} \mathbf{f}_j$ where $a_{i,j} \geq 0$ for all j. Otherwise, there would be non-zero elements in some $\mathbf{h}_i$ between indexes $l+1$ and n that would be impossible to eliminate in $\sum_{i=1}^{h-1} w_i \phi(\mathbf{h}_i)$ given that $w_i > 0$ for all i.

Second, any linear combination of $\mathbf{f}_1$ and $\mathbf{f}_2$ with positive coefficients would result in a vector whose first l elements are positive and increasing. In contrast, the first l elements of Y are constant. And so from the fact that there does not exist $a_{i,j} > 0$ such that the first l elements of $\sum a_{i,j}\mathbf{f}_j$ are decreasing—in particular because the first k elements and next $l-k$ elements of $\sum_{j=3}^{h+2} a_{ij}x_j$ are periodic with length h—it follows that $a_{i,1}, a_{i,2} = 0$ for all $\mathbf{h}_i$.

Thus, we need only consider linear combinations of $\mathbf{f}_3$ through $\mathbf{f}_{h+2}$ with positive coefficients as candidates for $\mathbf{h}_i$. To this end, note that if a particular $\mathbf{f}_j$ has zero coefficient in all of $\mathbf{h}_1$ through $\mathbf{h}_{h-1}$, then $\sum_{i=1}^{h-1} w_i\phi(\mathbf{h}_i)$ will have zeros in every index congruent to $j \mod h$ and therefore cannot equal $\mathbf{y}$. Hence by the pigeonhole principle, in order to have $\sum_{i=1}^{h-1} w_i\phi(\mathbf{h}_i) = \mathbf{y}$ there must be some i such that $\mathbf{h}_i = \sum_{j=3}^{h+2} a_{i,j}\mathbf{f}_j$ with at least two coefficients being non-zero. However, in any linear combination $\sum_{j=3}^{h+2} a_{i,j}\mathbf{f}_j$ where $a_{i,j}, a_{i,j'} > 0$ for at least two distinct j, j', the elements in indexes $k+1$ to l will be greater than the elements in indexes 1 to k that are congruent to $j \mod h$ and $j' \mod h$. In contrast, the first l elements of $\mathbf{y}$ are constant. Hence, similar to the case of $\mathbf{f}_1$ and $\mathbf{f}_2$, there cannot exist $\mathbf{h}_i = \sum_{j=3}^{h+2} a_{i,j}\mathbf{f}_j$ and positive coefficients w_i such that $\sum_{i=1}^{h-1} w_i\phi(\mathbf{h}_i) = Y$. □

D Experimental details and further results

D.1 Experimental details and hyperparameters

For all experiments on MNIST, we used a convolutional architecture consisting of 3 convolutional layers followed by a fully-connected output layer. Each convolutional layer consisted of 32 3×3 filters and used sufficient padding so as to keep the layer's output the same shape as its input. All networks were trained on an NVIDIA Tesla K20c GPU for 5000 iterations with a batch size of 64 using stochastic gradient descent with an initial learning rate of 0.1 and a decay rate of 1E^{-6}. No significant hyperparameter tuning was applied. Images were normalized.

For the left and right plots in Figure 2, we report results averaged over 5 random trials and error bars corresponding to the standard deviation over these trials. For the center plot we simply computed the loss and accuracy over a linear path between a particular convolutional net and a single dropout version of itself. Specific to Figure 2, in applying dropout with probability p we randomly sample a subset of $\lfloor 32(1-p) \rfloor$ units and rescale these units by $1/(1-p)$ while setting the remaining units to zero. In the left plot, each trial consisted of sampling 20 such dropout networks and reporting the performance of the network achieving the lowest loss. Losses and accuracies in all plots were computed on a random batch of 4096 training images.

On CIFAR-10, we trained VGG-11 networks on an NVIDIA Titan X GPU for 300 epochs with SGD with a batch size of 128, with weight decay 5e-4, momentum 0.9, and an initial learning rate of 0.05 which is decayed by factor of 2 every 30 epochs. We used channel-wise dropout at all convolutional layers. The dropout rates are $p = 0.25$ at the first three layers and are $p = 0.5$ at the others. Ordinary dropout with $p = 0.5$ is used at every fully-connected layers except for the last one (the softmax layer).

D.2 Straight interpolation between two models

As demonstrated in Figure 5, a straight line interpolation between two noise stable model may incur *large* losses and *poor* accuracies. The models are the same as used in Figure 3.

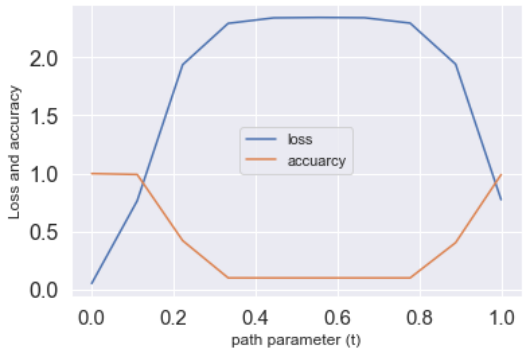
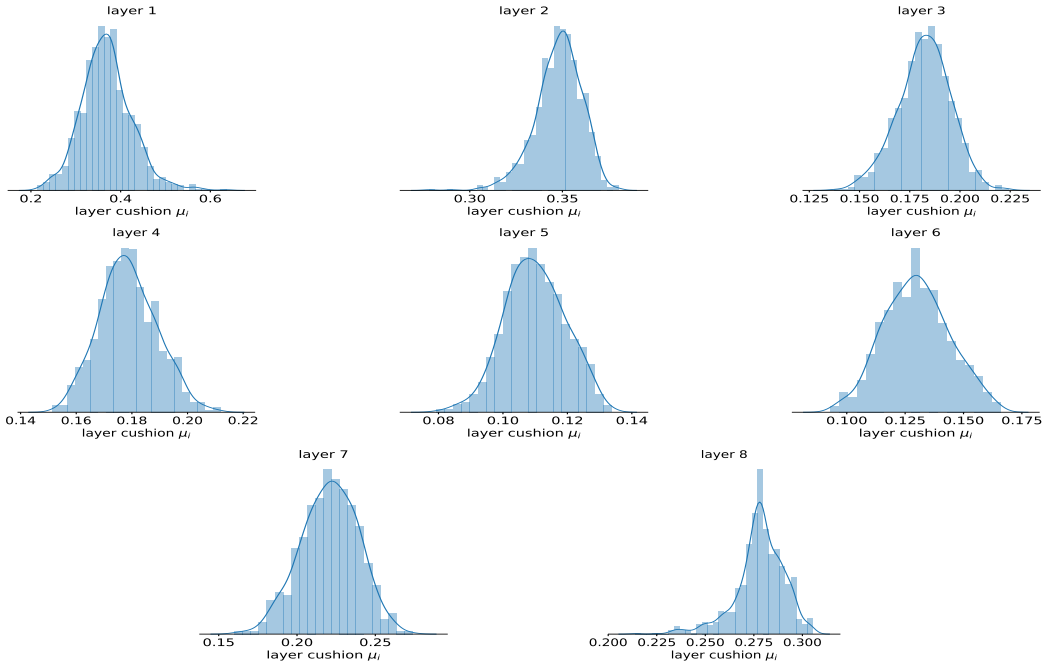


Figure 5: Loss and accuracy from directly interpolating between two noise stable models.

D.3 Verification of noise stability conditions

D.3.1 Layer cushion